

Détection automatisée des vulnérabilités des smart contracts Solidity : une approche basée sur l'apprentissage profond

NOTIONS

1. Blockchain et contrats intelligents

Une blockchain est un registre partagé dont les mécanismes de validation et les liens cryptographiques permettent de vérifier l'historique des opérations. Un contrat intelligent, ou smart contract, est un programme exécutant des règles sur cette infrastructure. La protection du registre ne garantit cependant pas la correction du programme : une erreur dans sa logique peut produire des comportements indésirables ou permettre une exploitation.

2. Solidity et vulnérabilités

Solidity est un langage utilisé pour développer des contrats destinés notamment à l'environnement Ethereum. Le corpus étudié regroupe des annotations relatives à la réentrance, aux appels externes non vérifiés, aux erreurs arithmétiques, au déni de service, à l'ordre des transactions, à la dépendance temporelle et à l'aléa insuffisant. Le système apprend un signal binaire global ; il ne fournit pas un diagnostic distinct de chacune de ces catégories.

3. Détection automatisée

La détection automatisée consiste ici à analyser le code source et à estimer si un contrat présente une vulnérabilité selon les annotations disponibles. Elle peut aider à prioriser les vérifications humaines. Une prédiction négative indique une absence de signal reconnu par le modèle ; elle ne constitue ni une preuve de sûreté ni une certification du contrat.

4. Apprentissage profond

L'apprentissage profond utilise des réseaux neuronaux pour apprendre des représentations à partir des exemples. Un CNN repère des motifs locaux, tandis qu'un BiLSTM traite une séquence dans les deux directions. Leur combinaison hiérarchique permet, dans ce projet, de représenter des fragments du code puis leurs relations à l'échelle du contrat.

5. Préentraînement et évaluation

Un modèle entraîné de zéro apprend ses paramètres sur le corpus de l'étude. Un modèle préentraîné réutilise des connaissances acquises auparavant. Ici, CodeT5 conserve son encodeur figé et seule une tête de classification est entraînée. La comparaison distingue l'entraînement, la validation, la calibration des scores et l'évaluation sur test.

Introduction générale

Les contrats intelligents permettent d'automatiser des opérations numériques et de faire exécuter des règles sans intervention continue de leurs concepteurs. Leur utilisation dans les applications décentralisées rend la qualité du code particulièrement importante. Lorsqu'un programme manipule des ressources ou commande des opérations irréversibles, une vulnérabilité peut avoir des conséquences difficiles à corriger après son déploiement.

La vérification des contrats Solidity repose sur plusieurs approches, notamment l'audit manuel, l'analyse statique et l'exécution symbolique. L'apprentissage profond apporte une démarche complémentaire : apprendre, à partir de contrats annotés, des régularités associées à un signal de vulnérabilité. Son intérêt ne peut toutefois être évalué uniquement à partir d'un score élevé. Les annotations, les doublons, les familles de contrats et la quantité de code réellement analysée influencent directement les résultats.

La problématique de ce mémoire est donc la suivante : dans quelle mesure une approche fondée sur l'apprentissage profond peut-elle contribuer à détecter automatiquement les vulnérabilités de contrats Solidity, et quels effets la conservation du code complet et le choix de l'architecture exercent-ils sur les performances observées ? Cette question implique de comparer les modèles selon un protocole commun et de distinguer la capacité à repérer les contrats vulnérables du nombre de fausses alertes.

L'objectif général est de concevoir et d'évaluer une chaîne de détection intégrée à l'application SMART BUG. Les objectifs spécifiques sont de préparer un corpus exploitable, de limiter les recouvrements entre partitions, de construire un CNN-BiLSTM hiérarchique, puis de le comparer à des réseaux simples, à une référence classique et à un modèle préentraîné. L'étude examine également la calibration des scores, la stabilité entre graines aléatoires et le coût d'inférence.

Le travail repose sur une classification binaire supervisée. Il ne vise ni à démontrer formellement l'absence de vulnérabilité ni à localiser avec certitude une ligne fautive. Les résultats sont interprétés comme des observations expérimentales sur les données retenues. Le test SMART BUG ayant déjà été consulté au cours du développement, la comparaison conserve un caractère exploratoire et nécessite une confirmation indépendante.

Le mémoire s'organise en trois chapitres. Le premier présente les concepts théoriques et les travaux proches. Le deuxième expose la méthodologie, la conception du système et la modélisation des données. Le troisième décrit la réalisation, l'expérimentation et les résultats, avant d'en discuter les limites. La conclusion générale reprend les apports du travail et les perspectives de recherche.

Chapitre 1. Concepts théoriques de base

1.1. Notions de base

L'apprentissage supervisé associe chaque exemple à une annotation utilisée pendant l'entraînement. Dans cette étude, la classe positive correspond à une vulnérabilité annotée et la classe négative à une absence de signal dans les annotations retenues. Cette distinction est essentielle : les labels dépendent des sources du corpus et ne représentent pas nécessairement une expertise exhaustive de chaque contrat.

Le code est transformé en tokens, c'est-à-dire en unités lexicales, puis en identifiants numériques. Des embeddings en apprennent une représentation vectorielle. Le CNN extrait des caractéristiques locales ; le BiLSTM modélise des dépendances séquentielles. L'architecture hiérarchique proposée applique ces mécanismes à deux niveaux : les tokens d'une fenêtre et la succession des fenêtres du contrat.

1.2. Revue de littérature

Tann et ses coauteurs proposent une approche séquentielle par LSTM pour détecter des menaces dans les contrats intelligents [1]. Leur travail constitue un antécédent de l'utilisation de réseaux récurrents dans ce domaine. Il montre que l'apprentissage sur des représentations séquentielles est une piste déjà étudiée ; le simple emploi d'un LSTM ne peut donc pas être présenté comme une nouveauté propre au mémoire.

Zhang et ses coauteurs présentent CBGRU, une méthode hybride associant des représentations de mots et plusieurs familles de réseaux neuronaux pour détecter des vulnérabilités [2]. Ce travail est proche par l'idée de combiner des caractéristiques complémentaires. Il motive une comparaison entre réseaux simples et modèle hybride plutôt que l'hypothèse d'une supériorité automatique de la combinaison.

Zhuang et ses coauteurs représentent les fonctions sous forme de graphes et appliquent des réseaux neuronaux de graphes à la détection de vulnérabilités [3]. Cette famille de méthodes exploite des relations structurelles différentes d'une simple séquence de tokens. Elle constitue une perspective pertinente pour intégrer plus explicitement les dépendances du programme.

Wang et ses coauteurs proposent CodeT5, un modèle préentraîné de compréhension et de génération de code tenant compte des identifiants [4]. Dans notre comparaison, CodeT5-small est utilisé comme extracteur de caractéristiques figé. Cette expérience mesure un transfert vers les données normalisées du projet, et non un fine-tuning complet sur Solidity.

Ces travaux situent la contribution du mémoire : une implémentation et une évaluation contrôlée du traitement du code complet, avec audit des données, calibration indépendante et comparaison de plusieurs architectures. Les scores publiés sur d'autres corpus ne sont pas directement comparables aux nôtres. Aucune nouveauté mondiale de l'architecture n'est revendiquée.

Chapitre 2. Méthodologie / Conception / Modélisation

2.1. Méthodologie

La démarche combine une analyse des travaux existants et une expérimentation reproductible. Les premières vérifications du projet ont mis en évidence des risques liés aux doublons, aux annotations contradictoires et à la troncature du code. Le protocole SMART BUG a donc été construit pour améliorer la préparation des données et séparer explicitement les différentes étapes d'apprentissage et d'évaluation.

Le corpus provient de plusieurs sources historiques, dont SmartBugs, Slither, DAppSCAN, SCRAWLD, Zeus et CGT, selon les catégories enregistrées dans le projet. Les exemples contradictoires ou insuffisamment exploitables sont mis en quarantaine. Les doublons sont traités et les contrats apparentés sont regroupés avant la répartition. Ces contrôles réduisent les recouvrements identifiés sans garantir l'absence de tous les clones approximatifs.

Le snapshot SMART BUG contient 18 238 exemples répartis entre cinq partitions. Les quatre premières servent au protocole principal ; la cinquième réserve une source afin d'examiner le comportement du système sur une provenance différente. Les partitions sont identiques pour toutes les variantes comparées [5].

Partition	Effectif	Rôle
Entraînement	11 959	Apprentissage des paramètres
Validation	1 709	Choix des checkpoints et de la variante
Calibration	1 709	Ajustement de la température et des seuils
Test	1 709	Évaluation exploratoire après sélection
Source réservée	1 152	Diagnostic de transfert entre sources

Le vocabulaire est construit uniquement sur l'entraînement. Les transformations apprises, notamment la standardisation des caractéristiques CodeT5, suivent la même règle. Une pondération des exemples selon leur source et leur classe est utilisée pendant l'apprentissage afin de limiter l'influence disproportionnée de certaines catégories du corpus.

Pour chaque entraînement, le checkpoint est choisi selon la log-loss de validation. La variante est ensuite sélectionnée selon le rappel moyen obtenu sur la courbe ROC de validation à un taux de faux positifs inférieur ou égal à 10 %, puis selon le F1 macro et la log-loss en cas d'égalité. Cette décision est enregistrée avant l'évaluation sur le test.

La calibration utilise une partition distincte. Une température ajuste les probabilités et deux seuils sont étudiés : l'un vise 90 % de rappel, l'autre au plus 10 % de faux positifs sur la calibration. Ces objectifs ne sont pas des garanties pour le test ou pour de nouveaux contrats.

2.2. Conception

La chaîne commence par l'analyse lexicale du code Solidity. Les commentaires sont retirés, les identifiants et certains littéraux sont normalisés, puis les tokens sont encodés. Le mode complet conserve l'ensemble de la séquence normalisée grâce à un découpage en fenêtres. Le mode tronqué conserve les 512 premiers tokens lexicaux avant encodage. Ainsi, « code complet » désigne la couverture de la séquence, pas la conservation littérale de tout le texte original.

Le CNN-BiLSTM hiérarchique associe une représentation locale et une représentation globale. Chaque fenêtre passe par une couche d'embedding et une convolution. Une agrégation par moyenne et maximum résume les caractéristiques de la fenêtre. Un BiLSTM traite ensuite la succession de ces représentations ; une nouvelle agrégation et une couche dense produisent un score pour le contrat. Le remplissage des lots est masqué afin de ne pas modifier les prédictions.

Les variantes CNN et BiLSTM permettent d'examiner l'apport de chaque famille de réseau. Le CNN agrège les caractéristiques de toutes les fenêtres ; le BiLSTM seul parcourt la séquence de tokens. La référence TF-IDF utilise des unigrammes et bigrammes, jusqu'à 20 000 caractéristiques, suivis d'une régression logistique. Elle constitue un point de comparaison classique pour apprécier l'utilité des réseaux profonds.

Pour CodeT5-small, tous les sous-tokens sont traités dans des fenêtres de 256 positions, tokens spéciaux compris. La moyenne pondérée et le maximum des états de l'encodeur constituent les caractéristiques du contrat. Celles-ci sont standardisées sur train, puis une tête dense est entraînée. Les poids de l'encodeur restent fixes : les trois graines font varier uniquement l'apprentissage de cette tête.

2.3. Modélisation

Les réseaux depuis zéro utilisent un vocabulaire de 8 000 entrées et des embeddings de dimension 32. Les convolutions comportent 24 filtres de taille 5 ; les LSTM disposent de 24 unités par direction. La couche dense contient 32 unités et le dropout vaut 0,3. L'optimiseur Adam démarre avec un taux d'apprentissage de 0,001. L'apprentissage est limité à 12 époques, avec arrêt anticipé après trois époques sans amélioration de la log-loss de validation.

Les lots contiennent au plus 32 contrats, avec un budget de 65 536 positions après remplissage, sauf contrat individuel plus long. Les graines 42, 73 et 101 servent à mesurer la variabilité. Le budget est comparable en passages sur les données, mais pas en nombre de paramètres, en opérations ou en temps CPU. Les checkpoints, configurations et empreintes des fichiers sont conservés pour documenter l'expérience.

Le rappel mesure la part des contrats positifs retrouvés ; le taux de faux positifs mesure la part des négatifs signalés à tort. Le F1 macro combine les F1 des deux classes avec un poids égal. Le score de Brier et l'ECE évaluent la qualité des probabilités, tandis que la latence renseigne sur le coût d'utilisation du modèle.

Chapitre 3. Simulation / Expérimentation / Réalisation

3.1. Expérimentation et réalisation

La mise en œuvre est une expérimentation sur des contrats annotés, et non la simulation d'un réseau blockchain. Elle comprend 18 entraînements depuis zéro, six entraînements de têtes CodeT5 et deux ajustements TF-IDF, soit 26 exécutions. Les nouvelles variantes neuronales sont entraînées sous PyTorch sur CPU. L'application SMART BUG conserve son modèle TensorFlow actif ; les essais comparatifs ne l'ont pas remplacé.

L'application permet de soumettre du code Solidity et d'obtenir une prédiction issue de la chaîne d'inférence. Les tests du projet vérifient notamment le masquage, la conservation de la fin des contrats, la restauration des checkpoints et l'intégration de la prédiction à l'API. Les 32 tests actuels passent. Les empreintes des 26 modèles et l'alignement des prédictions avec les labels ont également été contrôlés.

3.2. Résultats

Le tableau présente les résultats sur le test SMART BUG déjà consulté, pour les variantes à code complet. Les réseaux sont résumés par la moyenne sur trois graines ; TF-IDF correspond à un seul ajustement. Les seuils sont fixés sur calibration avec une cible de rappel de 90 % [5].

Modèle	F1 macro (%)	Rappel (%)	Faux positifs (%)	Latence (ms)
CNN-BiLSTM	84,54	89,07	19,84	3,00
CodeT5 figé	83,40	86,18	19,30	558,48
CNN	83,36	87,97	21,07	1,74
BiLSTM	83,10	88,16	21,76	6,97
TF-IDF	81,39	84,20	21,34	2,42

Le CNN-BiLSTM complet obtient le meilleur F1 moyen observé sur le test : 84,54 %, avec un rappel de 89,07 %. CodeT5 atteint 83,40 % de F1 et 86,18 % de rappel. La latence médiane est d'environ 3,00 ms pour le CNN-BiLSTM et 558,48 ms pour CodeT5. Ces mesures incluent l'encodage propre au modèle, mais excluent le chargement des poids, les accès disque et la normalisation Solidity commune.

La conservation du code complet produit un effet marqué : pour le CNN-BiLSTM, le F1 passe de 74,85 % avec l'entrée tronquée à 84,54 %, soit un gain de 9,69 points. Les cinq familles de modèles présentent un meilleur F1 moyen avec le code complet dans cette expérience. Cet effet concerne les données et les annotations étudiées ; il ne garantit pas la même amplitude sur un nouveau corpus.

3.3. Limites et améliorations possibles

La sélection sur validation et les observations sur test doivent être distinguées. Selon le critère fixé, CodeT5 complet est la variante retenue sur validation. Le CNN-BiLSTM obtient ensuite le meilleur F1 moyen observé sur le test. Ce constat ne justifie pas de modifier rétroactivement la sélection à partir du test ; il montre qu'un classement peut varier entre partitions et entre critères.

L'écart de F1 moyen entre CNN-BiLSTM et CodeT5 est de 1,14 point. Son intervalle bootstrap apparié à 95 %, calculé par groupes de contrats, est d'environ [-0,26 ; +2,69] points et inclut zéro. Une supériorité générale du CNN-BiLSTM n'est donc pas établie. Le bootstrap repose sur 1 000 rééchantillonnages, avec les graines et la calibration exécutées fixées, sans correction des comparaisons multiples.

Le test SMART BUG ayant déjà été consulté, l'évaluation est exploratoire. Par ailleurs, la troncature introduit des préfixes identiques pour 386 exemples du test par rapport à au moins une partition d'entraînement, de validation ou de calibration. Un diagnostic commun sur les 1 323 exemples sans ces collisions donne un F1 de 84,32 % pour le CNN-BiLSTM complet et de 83,32 % pour CodeT5 complet. Cette restriction ne constitue pas une nouvelle réserve indépendante.

Les annotations historiques comportent une incertitude, et certaines sources dominent le corpus. La source réservée contient 1 146 exemples positifs pour seulement six négatifs, ce qui rend l'estimation des faux positifs très fragile. Le classifieur demeure binaire : il ne fournit pas une localisation certaine de la faille ni une explication causale du verdict. Une analyse humaine reste nécessaire pour interpréter les alertes.

Les résultats CodeT5 ne couvrent que l'utilisation d'un encodeur figé sur du code normalisé. Ils ne permettent pas de conclure qu'un modèle préentraîné serait systématiquement moins utile, ni de prévoir les performances d'un fine-tuning sur Solidity. Les durées d'entraînement incluent aussi des pauses de la machine et ne doivent pas être assimilées à des temps de calcul purs ; les latences ont été mesurées séparément.

Les améliorations prioritaires sont la constitution d'un test indépendant séparé par familles et projets, la vérification des labels et l'ajout de négatifs comparables dans les sources réservées. Un fine-tuning contrôlé de CodeT5, des représentations par graphes, une sortie par catégorie de vulnérabilité et des mécanismes d'explication peuvent ensuite être étudiés. Ces perspectives nécessitent de nouvelles expériences et ne sont pas présentées comme des fonctionnalités déjà validées.

Conclusion générale

Ce mémoire étudie la détection automatisée des vulnérabilités des smart contracts Solidity par apprentissage profond. Il aboutit à une chaîne de préparation des données, à un modèle CNN-BiLSTM hiérarchique et à une comparaison avec des réseaux simples, une référence classique et un modèle préentraîné. Le travail met en évidence l'importance de la couverture du code et de la qualité du protocole d'évaluation.

Le résultat le plus net est l'amélioration du F1 lorsque le code complet est conservé. Le CNN-BiLSTM complet atteint 84,54 % de F1 macro et 89,07 % de rappel sur le test exploratoire, avec une faible latence mesurée. CodeT5, sélectionné sur validation, n'obtient pas le meilleur F1 sur ce test et nécessite davantage de temps d'inférence. L'écart entre les deux méthodes demeure toutefois incertain ; ces résultats ne prouvent pas une supériorité universelle de l'architecture hybride.

La contribution réside ainsi dans une réalisation reproductible et une analyse comparée des choix de représentation et de modélisation. Les observations ne justifient pas le remplacement automatique du modèle actuellement actif dans SMART BUG. Elles invitent à conserver le traitement du code complet et à confirmer les résultats sur des contrats indépendants avant toute conclusion générale ou évolution du modèle déployé.

Références bibliographiques

- [1] Tann, W. J.-W., Han, X. J., Sen Gupta, S. et Ong, Y.-S. (2018, version révisée 2019). Towards Safer Smart Contracts: A Sequence Learning Approach to Detecting Security Threats. arXiv:1811.06632. Consulter la source.
- [2] Zhang, L. et al. (2022). CBGRU: A Detection Method of Smart Contract Vulnerability Based on a Hybrid Model. Sensors, 22(9), 3577. DOI : 10.3390/s22093577. Consulter la source.
- [3] Zhuang, Y. et al. (2020). Smart Contract Vulnerability Detection using Graph Neural Network. IJCAI, p. 3283-3290. DOI : 10.24963/ijcai.2020/454. Consulter la source.
- [4] Wang, Y., Wang, W., Joty, S. et Hoi, S. C. H. (2021). CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. EMNLP, p. 8696-8708. DOI : 10.18653/v1/2021.emnlp-main.685. Consulter la source.
- [5] Projet SMART BUG (2026). Protocole SMART BUG et expérience comparaison-v1-20260922 : configurations, audit, résultats et rapport. Dépôt GitHub, commit e7f9ae0. Résultats expérimentaux du présent mémoire. Consulter la source.